

INTRODUCTORY MATLAB

ABOUT MATLAB

- Interactive Software used in various areas of engineering and scientific applications.
- Not a computer language though it does most of the work of a computer language.
- Easy to learn.
- Does not require the in depth knowledge of computer programming like compiling and linking. (Can be looked upon as an advantage and also as a disadvantage).
- Plotting.
- Interactive mode may reduce computational speed.

COMMAND WINDOW

- When we run MATLAB by double clicking on MATLAB on the desktop, it creates one or more windows on the computer monitor. Out of these, COMMAND WINDOW is the place where we interact with MATLAB.
- The prompt `>>` is displayed in the command window and when the command window is active, a blinking cursor should appear on the right side of the prompt.
- Simple Computations can be performed by typing the command and pressing enter.

Example: $S = 1+2$, $\text{fun} = \sin(\pi/4)$

```
>> S=1+2;

>> fun = sin(pi/4)

fun =

0.7071
```

Try yourself:

$2 + \text{floor}(6 / 9 + 3 * 2) / 2 - 3$

COMMENTS, PUNCTUATION AND ABORTING EXECUTION

- *Semicolon* at the end of a command suppresses the printing of computed results.
- All text after a *percent sign (%)* is taken as a comment statement.

Multiple commands can be placed on one line if they are separated by *commas* or *semicolons*.

- *Commas* tell MATLAB to *display* results; *semicolon* suppress printing.
- To *continue* expressions or commands onto additional lines, we use *...* in succession. *Comment lines and variable names cannot be split between two lines*.
- MATLAB processing can be interrupted at any moment by pressing `Ctrl+C`.

COMPLEX NUMBERS

- No special handling for complex numbers.

```
% Complex numbers

c1=1-2i % the appended i signifies
the imaginary part

c1= 1-2j % also works
c1= 1-2*(sqrt(-1)) % also works
realpart_c1=real(c1) % gives real
part
imagpart_c1=imag(c1) % gives
imaginary part
mag_c1=abs(c1) % gives magnitude of
c1
angle_c1= angle(c1) % gives angle in
radians

z=3+i*3;
compass(z) % polar representation
abs(z) % Modulus of complex number
angle(z) %argument of complex number

% Mathematical functions
x=sqrt(2) %gives square root of 2
y=rem(23,4) % remainder function,
23/4 has remainder 3

y_deg=30; %degree to be converted to
radians
y_rad=y_deg*pi/180;
y1=sin(y_rad)

x1=exp(2) %exponential function e^2
```

Try yourself:

$$(2+6*i)/(9*i)*(1+i)$$

MATHEMATICAL FUNCTIONS

- sqrt, sin, cos, sinh, asin, acos, exp, log etc.

Note: Can handle specialized mathematical functions Bessel, Legendre, Beta function etc.

NUMBER DISPLAY FORMATS

By default,

- if a result is an integer, it is displayed as an integer.
- if real number, four digits to right of decimal place is displayed.

```
% Number display formats:

formatshort;
pi = 3.14159
formatlong;
pi = 3.141592653589793
formatshorte;
pi = 3.14159e+000
formatlonge;
pi = 3.141592653589793e+000
```

SIMPLE ARRAY

- Start with a left bracket, enter the desired values separated by spaces (or commas) and then close it using a right bracket.
- Colon notation

$x = \text{first}:\text{last}$

$x = \text{first}:\text{increment}:\text{last}$

- linspace command

$x = \text{linspace}(\text{first}, \text{last}, n)$

Note: These arrays have one row and multiple columns.

```

% ARRAY CONSTRUCTION

% Method 1.
x=[0, 0.1*pi,0.2*pi,0.3*pi,
0.4*pi,0.5*pi,0.6*pi,0.7*pi,0.8*pi,
0.9*pi,pi]
y=sin(x)

% ARRAY INDEXING

x(3) % gives 3rd element of the
array

x(1:5) % access elements from 1
to 5

x(7:end) %starts with 7th and
continues till last element

y(3:-1:1) %start with third
element, count down by 1, stop at
1
% gives third, second and first
element

x(2:2:7) % start with second
element, count up by 2, stop when
you get 7
%Method 2

x1=3:20 % takes values from 3 to
20 incrementing 1

%Method 3

x= (0:0.1:1)*pi % 0 to pi in
increments of 0.1*pi

% Method 4
echoon
x=linspace(0,pi,11) ;
y=sin(x)
%linspace(first_value,last_value,
number of values)
echooff

```

Try yourself:

Given the array $A = \begin{bmatrix} 2 & 4 & 1 \\ 6 & 7 & 2 \\ 3 & 5 & 9 \end{bmatrix}$, provide the commands needed to

- assign the first row of A to a vector called x1
- assign the last 2 rows of A to an array called y
- compute the sum over the columns of A

COLUMN VECTORS

- Specify it element by element and separate values with semicolons.

Note: Separating elements by spaces or commas specifies elements in different columns, whereas separating elements by semicolons specifies elements in different rows.

- $b=a'$ transposes.
- $length(a)$ gives length of the vector 'a'.
- $a.*a$ is for component wise application of the operator a on a; if
 $a=[1 \ 2 \ 3];$
 $a.*a=[1 \ 4 \ 9]$
- $a.^m$ gives m^{th} power of components of a.
- $a./b$ gives component wise division of elements of a by elements in b.
- $a*b$ gives dot product of the vectors a and b.
- $cross(a,b)$ gives cross product of 2 three dimensional vectors.
- $unique(a)$ gives the same values as in array a but without repetitions.
- $sort(a)$ sorts a in ascending order, for matrices sorts columns in ascending order.
- $setdiff(a,b)$ returns vector of values in a but not in b.
- $max(a);min(a)$ gives maximum and minimum component of vector 'a'.
- $mean(a);std(a)$ gives mean and standard deviation of components.

```
% Vector operations

echoon
a= [1 4 1];
b=[6; 7; 8];
% transpose
c=a'
% length of a vector
length(a)
% component wise power
d= a.^(0.5)
% component wise square
e=a.*a
% dot product
f=a*b
% component wise division
f= a./b'
% cross product
g=cross(a,b)
% unique- gives the values of
array without repetitions
uni=unique(a)
% sorting in ascending order
ascend=sort(a)
% setdiff gives elements in a not
in b
set= setdiff(a,b')
echooff
```

Try yourself:

Let $x = [3 \ 2 \ 6 \ 8]'$ and $y = [4 \ 1 \ 3 \ 5]'$.
Add x and y and then find sum of elements of
resulting vector.

MATRICES OPERATIONS

Formation of matrices:

- Commas or spaces are used to separate elements in a specific row and semicolon are used to separate individual rows.
- Alternately, pressing return or enter key while entering a matrix also tells MATLAB to start a new row.

Example: $A = [1 \ 2 \ 3 \ 4; \ 5 \ 6 \ 7 \ 8; \ 1 \ 1 \ 2 \ 3]$ denotes the matrix with first row as 1 2 3 4, second row as 5 6 7 8, third row as 1 1 2 3.

- To extract a submatrix B consisting of row 1 and 3, columns 2 and 4, we use the command $B = A([1 \ 3], [2 \ 4])$.
- To interchange rows 1 and 3 of A use the row indices together with colon operator: $C = A([3 \ 2 \ 1], :)$. The colon operator $:$ stands for all columns or all rows.
- $A(:)$ creates a vector version of a matrix A .
- To delete a row(column) use the empty vector operator $[]$.

Example: $A(:, 2) = []$.

- To insert the second column back.

Example: $A = [A(:, 1) \ [2 \ 6 \ 1] \ A(:, 3) \ A(:, 4)]$

- Let $d = [1 \ 2 \ 3]$; $D = \text{diag}(d)$ is a diagonal matrix with diagonal entries in d .

```
% Matrix operations
%
echoon
A=[1 2 3 4; 5 6 7 8; 9 10 11 12]

% To interchange rows 1 and 3 of A
C=A([3 2 1], :)

% To create the vector version of A
D=A(:)

% To delete column 2 of A
A(:, 2) = []

%To find diag of A
D=diag(A)
echooff
```

Scalar Array Mathematics:

- Addition, subtraction, multiplication(.*), division by a scalar(/) applied to all elements in the array/ matrix.

Array – Array Mathematics & Matrix Functions

size(A)	order of matrix
A'	transpose of matrix
A(:,3)	third column of A
A(1,:)	first row of A
A(1,:)+A(3,:)	first row+third row of A
C=A+B	addition of matrices
C=A*B	multiplication of matrices
inv(A)	inverse of matrix
chol(A)	cholesky decomposition of a positive definite matrix A
det(A)	determinant of matrix
rank(A)	rank of matrix
eye(n)	identity matrix of order n
trace(A)	trace of matrix
zeros(n,m)	nxm zero matrix
rand(n,m)	nxm matrix having random entries

```
%Computingcputime for the program.

T=cputime;

A=[1 3 6; 2 7 8; 0 3 9]

%Order of A can be found out
order= size(A)

%Transpose of A
B=A'

%3rd column of A
thirdcolumn=A(:,3)

%First row of A
firstrow=A(1,:)

%C=A+B adds the matrices A and B
C=A+B

%D=A*B gives the product of the
matrices A and B
D=A*B

%A^-1
invA=inv(A)

%Determinant of A
detA=det(A)

%Rank of A
rank(A)

%Trace of A gives the sum of diagonal
elements of A
tr=trace(A)

%Identity matrix of order 3 is
generated now
identity=eye(3)

%Generating a zero matrix of order 2
by 3
zero=zeros(2,3)

%Matrix of order 3 x 4 with random
entries
Random=rand(3,4)
```

diag(v)	produces a diagonal matrix whose elements are components of vector v
eig(A)	eigen values of matrix A
[V,D]=eig(A)	produces V matrix whose columns are eigenvectors and the diagonal matrix D whose values are eigenvalues of the matrix A
$A=VDV^{-1}$; V^*D/V	eigenvalue-eigenvector decomposition of a diagonalizable matrix A is calculated
[L,U]=lu(A)	LU decomposition
[Q,R]=qr(A)	QR decomposition

```
%Generating a diagonal matrix.
v=[3 2 1 8];
M=diag(v)

% Extracting lower and upper triangular
parts of a matrix.
B=[3 1 2 ; -1 3 4 ; -2 -1 3]
L= tril(B)
U = triu(B)

%Eigenvalues of matrix A
eigv=eig(A)
[V,D]=eig(A)

% Extracting orthonormal and upper
triangular parts of a matrix.
[Q,R]=qr(A)
Q*Q'
```

Try yourself:

Generate two random matrices of order 3x3 and find their product and inverse of product.

Solution of linear equations : $Ax=y$

Assuming the system has a unique solution, we can obtain x as:

- $x=\text{inv}(A)*y$ (computes inverse explicitly)
- $x=A\backslash y$ (uses LU decomposition)

```
%Solving Ax=y
A=[1 3 4; 5 7 8; 2 3 5]
y=[10; 9; 8]

%computing inverse of a matrix A
Inverse=inv(A)
Identity=A*inv(A)
x=inv(A)*y
x1=A\y
```

Computing the CPU time (in seconds):

- $T=cputime$;
- Operations....
- $TT=cputime-T$;

LOOPS AND LOGICAL STATEMENTS:

Multiple for loops :

```
for i=1:100
    for j=1:50
        for k=1:50
            a(i,j)=b(i,k)*c(k,j)+a(i,j);
        end
    end
end
```

While

```
while( condition)
    statements
end
```

if, elseif, else, end

if(condition #1)

statement #1

elseif(condition #2)

statement #2

else

statement #3

end

Try yourself:

Given $x = 1:10$ and $y = [3 \ 1 \ 5 \ 6 \ 8 \ 2 \ 9 \ 4 \ 7 \ 0]$, execute and interpret the results of the following commands:

1. $x(x < 2) \mid (x \geq 8)$
2. $y(x < 2) \mid (x \geq 8)$
3. $x(y < 0)$.

PREALLOCATING ARRAYS

for and while loops that incrementally increase the size of a data structure each time through the loop can adversely affect performance and memory use. Repeatedly resizing arrays often requires that MATLAB spend extra time looking for larger contiguous blocks of memory and then moving the array into those blocks. You can often improve on code execution time by preallocating the maximum amount of space that would be required for the array ahead of time. The following code creates a scalar variable x , and then gradually increases the size of x in a for loop instead of preallocating the required amount of memory at the start:

$x = 0;$

for $k = 2:1000$

$x(k) = x(k-1) + 5;$

end

Change the first line to preallocate a 1-by-1000 block of memory for x initialized to zero.

This time there is no need to repeatedly reallocate memory and move data as more values are assigned to x in the loop:

$x = \text{zeros}(1, 1000);$

for $k = 2:1000$

$x(k) = x(k-1) + 5;$

end

VECTORIZATION

One way to make your MATLAB programs run faster is to vectorize the algorithms you use in constructing the programs.

A simple example involves creating a table of logarithms.

$x = 0.01;$

for $k = 1:1001$

$y(k) = \log_{10}(x);$

$x = x + .01;$

end

A vectorized version of the same code is:

$x = .01:.01:10;$

$y = \log_{10}(x);$

```

% Loops and logical statements

%for loop
%-----

%Compute the first 6 terms of the
Fibonacci sequence

%f_i=f_{i-1} + f_{i-2}
%echo on
f(1)=0; f(2)=1;
for i=[3 4 5 6] %for i=3:6
f(i)=f(i-2)+f(i-1);
end
break

% Ex 2 for for loop
%-----
% Generating sine function for 11 evenly
placed points

for n=0:10
x(n+1)=sin(pi*n/10);
end

%break
% Ex3 for for loop-nested for loops
%-----
% Creating the Hilbert matrix.

H=zeros(5);
for k=1:5;
for l=1:5;
H(k,l)=1/(k+l-1);
end
end

%The first co%A=[a_{kl}], where
%a_{kl}=sin(k) cos(l). We will compute
entries of A by using nested loops
% and also by vectorization.

A=zeros(10);
for k=1:10
for l=1:10
A(k,l)=sin(k)*cos(l);
end
end

```

```

% A loop free version-vectorization is
as follows:
%-----
k=1:10;
A=sin(k) '*cos(k);

%while loop
%-----

f(1)=0; f(2)=1; k=3;
while k <=6
f(k)=f(k-1)+f(k-2);
k=k+1;
end
% if / else if
%-----
% Roots of the polynomial ax^2+bx+c
a=input('Enter value of a =');
b=input('Enter value of b =');
c=input('Enter value of c =');

if a~=0
sq=sqrt(b*b-4*a*c);
x(1)=0.5*(-b+sq)/a
x(2)=0.5*(-b-sq)/a
elseif b~=0
x(1)=-c/b
elseif c~=0
disp('Impossible equation')
else
disp('The given equation is an
identity')
end

```

FUNCTION SUBROUTINES

- Special type of m file.
- Receives input data through input argument list.
- Returns results to the caller through an output argument list.


```
%The distance between two points in 2D
plane.

%Definition of variables:
%ax= x position of the point a
%ay= y position of the point a
%bx= x position of the point b
%by= y position of the point b
%result=distance between two points

%Get input data
disp('Calculate the distance between two
points')
ax=input('Enter x value of point a: ');
ay=input('Enter y value of point a: ');
bx=input('Enter x value of point b: ');
by=input('Enter y value of point b: ');

%Evaluate function
result=dist2(ax,ay,bx,by)
*****
function distance=dist2(x1,y1,x2,y2)
%Output argument is distance which is
the distance between (x1,y1)and (x2,y2)
distance=sqrt((x2-x1).^2+(y2-y1).^2);
%Notice that the function has 4 input
arguments and 1 output argument.
```

Try yourself:

Calculate distance between x =(1,4) and y=(5,9).

PLOTTING TOOLS (2D)

Plot(x,y,'r')	co-ordinates x-y, colour-red
axis([a,b,c,d])	dimension of the box holding the plot [a b c d]=[x _{min} x _{max} y _{min} y _{max}]
xlabel(' ')	label x axis
ylabel(' ')	label y axis
legend(' ')	distinguish various curves in same plot
grid	adds grid lines to graph
pause()	holds on to current graph till you press enter
Pause(n)	holds on to current graph for n seconds

SUBPLOTS

Subplot can be used for plotting more than one set of axes in a single figure. Subplots are created with the help of the command

subplot(m,n,p)

This creates m x n subplots in the current figure, arranged in m rows and n columns, and selects subplot p to receive all current plotting commands.

Example: subplot(2,3,4)

This will create six subplots in the current figure and will make subplot 4 (the lower left one) as the current subplot.

```
%Plotting graph using SUBPLOT

%function y = x/(1+x^2)

k = 0;
for n=1:3:10
    n10 = 10*n;
    x = linspace(-2,2,n10);
    y = x./(1+x.^2);
    k = k+1;
    subplot(2,2,k)
    plot(x,y,'r')
    title(sprintf('Graph %g. Plot based
upon n = %g points.'...
                , k, n10))
    xlabel('x')
    ylabel('y')
    axis([-2,2,-.8,.8])
    grid
    pause(3);
end
```

Try Yourself:

Plot $y = \sin x$, $0 < x < \pi$ taking 100 linearly spaced points in the given interval.

```

% Graphs of two ellipses in the same
%figure

% x(t) = 3 + 6cos(t), y(t) = -2 +9sin(t)
% x(t) = 7 + 2cos(t), y(t) = 8 +6sin(t)

% There are several commands to enhance
the readability of the graph.

%First of all note that the equations are
converted into parametric form.

t = 0:pi/100:2*pi;
x1 = 3 + 6*cos(t);
y1 = -2 + 9*sin(t);
x2 = 7 + 2*cos(t);
y2 = 8 + 6*sin(t);
h1 = plot(x1,y1, 'r',x2,y2, 'b');

% Plot colours: y=yellow, m = magenta,
c=cyan, r=red, g=green,b=blue, w=white,
k=black.

set(h1, 'LineWidth',1.25)
% To set the thickness of the plotted
curves to a width of our choice.

axis('square')
% To force the axes to have square
dimensions or else circular curves maynot
necessarily look circular.

legend('ellipse1', 'ellipse2')
xlabel('x')
h = get(gca, 'xlabel');
%gca= get current axis; the axis targetted
%here is x-axis.

set(h, 'FontSize',12)
set(gca, 'XTick',-4:10)
% Specify font size and tick marking along
the current axis.

ylabel('y')
h = get(gca, 'ylabel');
set(h, 'FontSize',20)
set(gca, 'YTick',-12:2:14)
title('Graphs of (x-3)^2/36+(y+2)^2/81 = 1
and (x-7)^2/4+(y-8)^2/36 = 1.')
h = get(gca, 'Title');
set(h, 'FontSize',12)
grid

```

subplot	creates an array of plots in same window
loglog	plot using log-log scales
semilogx	plot using log scale on the x-axis
semilogy	plot using log scale on the y-axis
surf	3-D shaded surface graph
surfl	3-D surface graph with lighting
mesh	3-D mesh surface

PLOTTING TOOLS (3D)

plot3(x,y,t)	3D plot
set(h,'LineWidth',1.25)	set linewidth
set(h,'FontSize',11)	set fontsize
colormap {colour}	to apply desired colour
shading interp	shading after interpolation
view([25,15,20])	To set viewing angle

ILLUSTRATION OF MESH GRID:

```

% Illustration of meshgrid
x=-1:0.05:1;
y=x;
[xiyi]=meshgrid(x,y);
zi=yi.^2-xi.^2;
mesh(xi,yi,zi)
axisoff
% To plot the graph of the mesh surface
along with contour plot beneath the
% plotted surface:
meshc(xi,yi,zi)

```

POLYNOMIALS

polyval	evaluates polynomial
roots(p)	computes zeros of $p(x)$
poly(r)	reconstructs polynomials from the roots
conv(p1,p2)	gives the coefficients of the polynomial given by the product of two polynomials whose coefficients are contained in the vectors $p1$ and $p2$.
polyder(p)	gives the coefficient of $p'(x)$
polyint(p)	gives coefficients of integral of $p(x)$

```
% Polynomials.....

% Values:
p=[1 0 0 0 0 3 0 -1]; % polynomial
x^7+3x^2-1
x=[-1:0.25:1]; % equispaced abscissae
% x_k=-1+k*0.25 for k=0,...8.
y=polyval(p,x) % Values of polynomial

% Zeros:
p=[1 -6 11 -6]; format long; % x^3-
6x^2+ 11 x -6
roots(p)

%Coefficients of polynomial p1*p2
p1=[1 0 0 0 -1] % p1(x)=x^4-1
p2=[1 0 0 -1] % p2(x)=x^3-1
p=conv(p1,p2) % Gives coefficient of
p1*p2

% Coefficients of p'(x), integral of
p(x)

derivative=polyder(p1)
integral=polyint(p1)
```

Try yourself:

Solve the polynomial $x^3 - 6x^2 + 11x - 6$ and calculate its first derivative and roots of the derivative.

Fzero FOR ZEROS OF NON-LINEAR EQUATIONS

- $fzero(fun,x0)$ adopts Dekker-Brent method to compute the zero of the function fun , where fun can be defined as an inline function or in a .m file.

Syntax:

$f=inline('fun','variable');$ $x0=..$

$[alpha,res,flag,iter]=fzero(fun,x0)$

where $alpha$ =root, res =residual, $iter$ = number of iterations, $flag$ is negative means $fzero$ can't find the zero.

SYMBOLIC TOOLS

After defining a function,

diff(f)	Differentiation of f
int(f)	Indefinite integration of f
taylor(f,x,r)	Taylor expression of f till r terms about $x=0$

Try yourself:

Generate Taylor expression of $f(x)=\sin(x)$ upto 6 terms at $x=0$.

Numerical Integration (composite midpoint quadrature formula)

```
%Numerical integration using composite
midpoint formula:
```

```
%Illustration of the use of inline
functions
```

```
a=input('a=');
b=input('b=');
M=input('number of subintervals=');
```

```
%f='5'; %Integrating a constant
function
```

```
%f=inline('1+x','x')
% f=inline('x.^2-10.*x+15','x')
f=inline('sin(x)','x')
%f=inline('1./(1+x.^2)','x')
```

```
function Imp=midpointc(a,b,M,f)
H=(b-a)/M;
x=linspace(a+H/2,b-H/2,M);
```

```
fmp=feval(f,x);
```

```
Imp=H*sum(fmp)
```

Try yourself:

Solve using composite midpoint quadrature formula:

$$\int_{-2}^2 x^3 e^x dx$$

Gauss-Lobatto Quadrature Formula

- We use an inbuilt function **quadl** to compute an integral using Gauss-Lobatto quadrature formula.
- The function is defined as an *inline* object.
- Call **quadl(fun,a,b)**. (*fun*-function, *a*, *b* – limits of integration)
- The number of subintervals is not requested as it is automatically

computed so as to ensure that quadrature error is below 10^{-3} .

- quadl(fun,a,b,tol)** specifies a different tolerance level.

```
%Gauss-Lobatto quadrature rule
```

```
a=input('a='); %specify limits of
integration
b=input('b=');
```

```
%fun=inline('1./x','x') %MATLAB will
evaluate the expression component by
component on the vector
```

```
fun=inline('1./(1+x.^2)','x')
```

```
int=quadl(fun,a,b) %integrates fun
with tolerance 10^-3.
```

```
%tol=input('tol=')
```

```
%int1=quadl(fun,a,b,tol)
```

Try yourself:

Solve using Gauss Quadrature formula

$$\int_2^3 \frac{\cos 2x}{1 + \sin x} dx$$

Newton's method for Non-Linear Systems

- To compute the roots of nonlinear systems of the following form:

$$f_1(x_1, x_2, \dots, x_n) = 0,$$

$$f_2(x_1, x_2, \dots, x_n) = 0,$$

$$f_3(x_1, x_2, \dots, x_n) = 0,$$

$$\dots,$$

$$f_n(x_1, x_2, \dots, x_n) = 0,$$

where f_i are nonlinear functions, using Newton's method, use

$$zero = fsolve('fun', x0)$$

This allows the computation of one zero of the nonlinear system defined through fun starting from x0 as initial guess.

```
% Newton's method to solve a system of
nonlinear equations:

% x^2 + y^2=1
% sin(pi*x/2)+y^3=0

% Solutions are (0.4761, -0.8794) and (-
0.4761,0.8794)

x0=[1 1]; % Initial iterate
alpha=fsolve('systemnl',x0)
```

General Instructions for Numerical Analysis Programs

- Declare the array variables to zero.
- Use clear all statement in the beginning of the code.
- Use long double for real arrays.
- Declare the matrix as “sparse” in case you are working with a sparse matrix to save memory and computation time. For ex. a sparse matrix of dimension 10* 100 is initialized by A = sparse(10,100). Huge identity matrices can be also declared as sparse matrix by using A=speye(100).
- A full matrix A can be transformed to a sparse matrix by using the command A=sparse(A).
- In case, a sparse matrix needs to be transformed to a full matrix, use the command A=full(A).
- Use vectorization as far as possible.
- Assign space in memory for arrays and matrices in the beginning.

ORDINARY DIFFERENTIAL EQUATIONS

- Write the differential equation (s) as a set of ODEs.

$$x'=f(x,t)$$

where $x = [x_1 \ x_2 \ \dots \ x_n]'$

- Use built-in ODE solvers 'ode23' or 'ode45' to solve the equations.

$[t, sol] = \text{ode23}('ur_func', tspan, x0)$

where :

t= time vector; sol= solution matrix;

tspan= time span; x0= initial conditions

Ode23	Uses Runge-Kutta scheme of order 2 and 3.
Ode45	Uses Runge-Kutta scheme of order 4 and 5.
Ode113	Uses explicit predictor - corrector method with variable order 1 and 13.
Ode15s	Uses Implicit multistep method with variable order 1 to 5.
Ode23s	Uses Runge-Kutta scheme of order 2 and 3.

```
% to solve the first order
differential equation dx/dt=x+t
% initial condition x(0)=0

function xdot=simpode(t,x)

%simpode : computes xdot=x+t

%call syntax : xdot=simpode(t,x)

xdot=x+t;
tspan=[0 2]; %specify timespan
x0=0; % specify x0

[t,x]=ode23('simpode',tspan,x0)
%now execute ode23
```

Try yourself:

Solve the following differential equation:

$$y' + 2xy^2 = 0; y(0) = 1$$

```
%to solve second order non linear
differential equation

%write a function to compute the new
state derivatives

function zdot=pend(t,z)

%call syntax: zdot=pend(t,z)
%inputs are: t=time

z=[z(1);z(2)]=[theta;thetadot]
%output is: zdot=[z(2);-w^2*sin z(1)]

wsq=1.56; %specify a value of w^2
zdot[z(2); -wsq*sin(z(1)) ]

%use ode23 ode 45 for solution :
tspan =[0 20];z0=[1;0]; %assign values to
tspan,z0
[t,z]=ode23('pend',tspan,z0) % run ode23
x=z(:,1);y=z(:,2);           %x=column 1
of z, y=column 2

plot(t,x,t,y)                 %plot t vs x
and t vs y

xlabel('t'),ylabel('x and y')%add axis
labels

figure(2)                     %open a new
figure window
plot(x,y)                     %plot phase
portrait
xlabel('displacement'),ylabel('velocity')
title('phase plane plot')     %put a title
```

EXERCISES

1. Let $x = [3 \ 2 \ 6 \ 8]$ and $y = [4 \ 1 \ 3 \ 5]$.
 - a) Add the sum of the elements in x to y .
 - b) Raise each element of x to the power specified by the corresponding element in y .

- c) Divide each element of y by the corresponding element in x .
 - d) Multiply each element of x by the corresponding element in y and call the result z .
 - e) Sum up the elements in z and assign the result to a variable called w .
 - f) Compute $(x-y)/w$ and interpret the result.
2. Evaluate the following expressions:
 - a) `round(6/9)`
 - b) `floor(6/9)`
 - c) `ceil(6/9)`
 3. Plot the functions x , x^3 , e^x and $e^{(x^2)}$ over the interval $0 < x < 4$.
 4. Given the array $A = [2 \ 4 \ 1; 6 \ 7 \ 2; 3 \ 5 \ 9]$, provide the commands needed to:
 - a) Assign the first row of A to a vector $x1 = [1 \ 2 \ 3]$.
 - b) Assign the last 2 rows of A to a matrix $Y = [2 \ 3 \ 1; 5 \ 3 \ 2]$.
 - c) Compute the sum over the columns of A and also over the rows of A .
 5. Given the vector $x = [1 \ 8 \ 3 \ 9 \ 0 \ 1]$, create a short set of commands that adds up the values of the elements of x .

6. Find the numerical solutions to the following initial value problems (IVP)
 - a) $dy/dt = 1/(t+y+1)$, $y(0)=0$, $0 < t < 1$
 - b) $dy/dt = \log \log(4+y^2)$, $y(0)=1$, $0 < t < 1$.
7. Find the approximate solution $x(1.1)$ of the initial value problem $dx/dt = 3t - x/t$; $x(1)=2$.